Cheat Sheet for the Final Exam

Name:	Student #:
Email:	UTorID :

Module 1: Introduction to Formal Languages

Alphabet (Σ): A finite, non-empty set of symbols. E.g., $\Sigma = \{0, 1\}$ or $\Sigma = \{a, b\}$.

Monoid: A set M with an associative binary operation $\star$ and an identity element e.

$$\forall a, b, c \in M, (a \star b) \star c = a \star (b \star c)$$

Examples: $(\mathbb{N}, +, 0)$, $(\mathbb{Z}_+, \times, 1)$.

Inclusion-Minimal Set: A set S is inclusion-minimal w.r.t property P if any set S' satisfying P contains S ($S \subseteq S'$).

The Free Monoid Σ^*: The set of all strings over Σ. Defined inductively as the inclusion-minimal set such that:

1. **Base:** $\epsilon \in \Sigma^*$ (empty string, length 0).
2. **Inductive:** If $a \in \Sigma$ and $u \in \Sigma^*$, then $au \in \Sigma^*$.

Structural Induction on Σ^*: To prove $\forall w \in \Sigma^*, P(w)$ is true:

1. **Base Case:** Prove $P(\epsilon)$ holds.
2. **Inductive Step:** Assume $P(u)$ is true (IH). Show $P(au)$ is true for all $a \in \Sigma$.

Recursive Definitions on Strings: Functions on Σ^* are often defined recursively.

- **Length $|w|$:**

$$|w| = \begin{cases} 0 & \text{if } w = \epsilon \\ 1 + |u| & \text{if } w = au \end{cases}$$

- **Concatenation $w \cdot v$:**

$$w \cdot v = \begin{cases} v & \text{if } w = \epsilon \\ a(u \cdot v) & \text{if } w = au \end{cases}$$

Lemma: $|w \cdot v| = |w| + |v|$.

- **Reversal w^R (or rev(w)):**

$$w^R = \begin{cases} \epsilon & \text{if } w = \epsilon \\ u^R a & \text{if } w = au \end{cases}$$

Formal Languages: A language L is a subset of Σ^* ($L \subseteq \Sigma^*$). Set operations apply ($A \cup B$, $A \cap B$, $A \setminus B$, A^c).

Language Concatenation:

$$KL = \{uv \mid u \in K, v \in L\}$$

Note: If $K = \emptyset$ or $L = \emptyset$, then $KL = \emptyset$.

Powers of Languages: Defined recursively:

$$L^k = \begin{cases} \{\epsilon\} & \text{if } k = 0 \\ LL^{k-1} & \text{if } k > 0 \end{cases}$$

Note: $L^0 = \{\epsilon\}$ (even if $L = \emptyset$). $\Sigma^k = \{w \in \Sigma^* : |w| = k\}$.

Kleene Star (L^*): The set of all strings formed by concatenating zero or more strings from L.

$$L^* = \bigcup_{n=0}^{\infty} L^n$$

Properties:

- $L \subseteq L^*$
- $\epsilon \in L^*$ (always, even for $\emptyset^*$)
- If $u, v \in L^*$, then $uv \in L^*$.
- L^* is the inclusion-minimal superset of L closed under concatenation and containing ϵ.

Examples:

- $\emptyset^* = \{\epsilon\}$
- $\{\epsilon\}^* = \{\epsilon\}$
- $\{0, 1\}^* = \{\epsilon, 0, 1, 00, 01, \dots\}$ (all binary strings)

Module 2: Mathematical Induction

Inductively Defined Sets: Defined as the *inclusion-minimal* set X satisfying specific properties (Base + Inductive clauses).

- $\mathbb{N}$: $0 \in \mathbb{N}$; if $x \in \mathbb{N} \to x + 1 \in \mathbb{N}$.
- Σ^*: $\epsilon \in \Sigma^*$; if $u \in \Sigma^*, a \in \Sigma \to au \in \Sigma^*$.
- **FBT (Full Binary Trees):**
  - Base: Single node tree(a) $\in$ FBT.
  - Ind: If $l, r \in$ FBT, then tree($a; l, r$) $\in$ FBT.

Structural Induction Principle: To prove $\forall x \in X, P(x)$:

1. **Base Case:** Prove $P(b)$ for all base elements $b \in X_0$.
2. **Inductive Step:** Prove that if $P(x_1), \dots, P(x_k)$ hold (IH), then $P(y)$ holds, where y is constructed from x_i.

Binary Tree Definitions: Recursive definitions on $t \in$ FBT:

- **Height $h(t)$:**

$$\begin{cases} 0 & \text{leaf} \\ 1 + \max(h(l), h(r)) & \text{tree}(a; l, r) \end{cases}$$

- **Size $n(t)$:**

$$\begin{cases} 1 & \text{leaf} \\ 1 + n(l) + n(r) & \text{tree}(a; l, r) \end{cases}$$

Theorem: $n(t) \leq 2^{h(t)+1} - 1$.

Noetherian (Well-Founded) Induction: Generalization of Strong Induction.

- **Quasi-order ($\preceq$):** Reflexive and transitive.
- **Well-Founded:** No infinite strictly decreasing chains $x_1 \succ x_2 \succ x_3 \dots$.
- **Principle:** To prove $P(x)$ for all $x \in A$: Show that $[\forall u \prec x, P(u)] \implies P(x)$.

(Base cases are implicit where the set of predecessors is empty).

Lexicographic Order: On $\mathbb{N} \times \mathbb{N}$: $(a_1, a_2) \preceq (b_1, b_2)$ iff: $a_1 < b_1$ or $(a_1 = b_1 \wedge a_2 \leq b_2)$. This is a well-founded quasi-order. Used for termination proofs (e.g., Ackermann function).

Module 3: Runtime Analysis

Asymptotic Definitions: Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$.

Big-O (Upper Bound): $f \in O(g)$ iff $\exists C > 0, n_0 \in \mathbb{N}$ s.t.

$$\forall n \geq n_0, f(n) \leq Cg(n)$$

Intuition: f grows no faster than g.

Big-Omega (Lower Bound): $f \in \Omega(g)$ iff $\exists C > 0, n_0 \in \mathbb{N}$ s.t.

$$\forall n \geq n_0, f(n) \geq Cg(n)$$

Intuition: f grows at least as fast as g.

Big-Theta (Tight Bound): $f \in \Theta(g)$ iff $f \in O(g)$ and $f \in \Omega(g)$.

$$C_1 g(n) \leq f(n) \leq C_2 g(n)$$

Limit Rule: Let $L = \lim_{n\to\infty} \frac{f(n)}{g(n)}$.

- If $L \in (0, \infty)$, then $f \in \Theta(g)$.
- If $L = 0$, then $f \in O(g)$ but $f \notin \Omega(g)$ ($f \prec g$).
- If $L = \infty$, then $f \in \Omega(g)$ but $f \notin O(g)$ ($g \prec f$).

Properties:

- **Transitivity:** $f \in O(g) \wedge g \in O(h) \implies f \in O(h)$.
- **Sum Rule:** $O(f + g) = O(\max(f, g))$.
- **Polynomials:** $\sum_{i=0}^{k} a_i n^i \in \Theta(n^k)$ (if $a_k > 0$).
- **Logarithms:** $\log_a n \in \Theta(\log_b n)$ (bases don't matter).

Hierarchy: $1 \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec 2^n \prec n!$

Time Complexity Cases: Let X_n be inputs of size n, $p(A; x)$ be steps for input x.

- **Worst Case:** $t(n) = \max_{x \in X_n} p(A; x)$.
- **Best Case:** $t_*(n) = \min_{x \in X_n} p(A; x)$.
- **Average Case:** $\bar{t}(n) = \frac{1}{|X_n|} \sum_{x \in X_n} p(A; x)$.

Recurrence Analysis Example: Harmonic series often appears in average case analysis:

$$H_n = \sum_{i=1}^{n} \frac{1}{i} \approx \ln n \implies H_n \in \Theta(\log n)$$

Geometric series $\sum_{i=0}^{n} r^i$:

- If $r > 1$, $\Theta(r^n)$.
- If $r = 1$, $\Theta(n)$.
- If $r < 1$, $\Theta(1)$.

Module 4: The Master Theorem

Recurrence Relations: Equations defining a function $f : \mathbb{N} \to \mathbb{N}$ in terms of its values on smaller inputs. Defined by: 1. **Base case(s):** Explicit values for small inputs (e.g., $f(0) = c$). 2. **Recursion:** $f(n)$ defined using $f(m)$ where $m < n$.

Solving via Back-Substitution: Method to find a closed-form expression. **Steps:** 1. Expand the recurrence repeatedly to find a pattern. 2. Express $t(n)$ in terms of k iterations:

$$t(n) = (\dots) + \sum(\text{cost at depth } i)$$

3. Determine k such that the recursion reaches the base case (e.g., $n/b^k = 1 \implies k = \log_b n$). 4. Solve the summation (often geometric or arithmetic series).

Simple Master Theorem: For a recurrence of the form:

$$t(n) = a \cdot t\left(\frac{n}{b}\right) + n^\ell$$

where $a \geq 1, b > 1, \ell \geq 0$. Compare a (rate of subproblem growth) with b^ℓ (rate of work shrinkage/growth).

Cases:

1. **Leaves Dominate ($b^\ell < a$):** Work increases geometrically down the tree.
$$t(n) \in \Theta(n^{\log_b a})$$
2. **Balanced Levels ($b^\ell = a$):** Work is constant per level (total work $\approx$ height $\times$ level cost).
$$t(n) \in \Theta(n^\ell \log_b n)$$
3. **Root Dominates ($b^\ell > a$):** Work decreases geometrically down the tree.
$$t(n) \in \Theta(n^\ell)$$

Extended Master Theorem: For the general recurrence:

$$t(n) = a \cdot t\left(\frac{n}{b}\right) + f(n)$$

Let $d = \log_b a$ (the "critical exponent").

Cases:

1. **$f(n)$ is polynomially smaller than n^d:** If $f(n) \in O(n^{d-\epsilon})$ for some $\epsilon > 0$:
$$t(n) \in \Theta(n^d)$$
2. **$f(n)$ is "similar" to n^d:** If $f(n) \in \Theta(n^d \log^k n)$ for $k \geq 0$:
$$t(n) \in \Theta(n^d \log^{k+1} n)$$
3. **$f(n)$ is polynomially larger than n^d:** If $f(n) \in \Omega(n^{d+\epsilon})$ for some $\epsilon > 0$, AND the regularity condition holds ($af(n/b) \leq Cf(n)$ for $C < 1$):
$$t(n) \in \Theta(f(n))$$

Important Identities:

- $a^{\log_b n} = n^{\log_b a}$
- Geometric Sum: $\sum_{i=0}^{k-1} r^i = \frac{1-r^k}{1-r}$
- Height of recursion tree: $k \approx \log_b n$.

Module 5: D&C Algorithms

Overview: Algorithms that break a problem into smaller instances.

- **Decrease-and-Conquer:** Reduces instance to *one* smaller version (e.g., Binary Search).
- **Divide-and-Conquer:** Breaks instance into *two or more* smaller instances (e.g., Merge Sort).

Algorithm Structure: Given a class of problems $\mathscr{P}$:

1. **Base Case ($\mathscr{P}_0$):** Small inputs where solution $s(P)$ is known explicitly.
2. **Reduction (F):** Reduces $X \in \mathscr{P}$ to smaller A (and B).
3. **Construction (G):** Constructs solution for X using solutions to A (and B).

Decrease-and-Conquer Template:

```
Func DecC(P):
  if P in P0: return s(P)
  A = F(P)        // Reduction
  A' = DecC(A)    // Recursion
  Q = G(A')       // Construction
  return Q
```

Reduction can be arithmetic ($n - c$) or geometric (n/c).

Example: Binary Search: Problem: Find x in sorted array L.

- **Base:** $|L| = 1$, check if element is x.
- **Reduction:** Compare x to $L[\text{mid}]$. Recurse on Left or Right subarray.
- **Construction:** None (return result directly).
- Complexity: $T(n) = T(n/2) + 1 \implies O(\log n)$.

Example: Map Coloring: Problem: 2-color a map formed by n lines.

- **Reduction:** Remove one line. Map has $n - 1$ lines.
- **Construction:** Flip colors on one side of the restored line.

Divide-and-Conquer Template:

```
Func DivC(P):
  if P in P0: return s(P)
  A, B = F(P)          // Split
  A' = DivC(A)
  B' = DivC(B)
  Q = G(A', B')        // Merge
  return Q
```

Key Examples:

- **Merge Sort:** Split list in half, sort recursively, merge sorted lists. $T(n) = 2T(n/2) + n$.
- **Karatsuba Mult:** Split integers into high/low bits. Uses 3 mults instead of 4. $T(n) = 3T(n/2) + n$.
- **Strassen's Matrix Mult:** Uses 7 recursive mults for 2×2 blocks instead of 8. $T(n) = 7T(n/2) + n^2 \implies O(n^{\log_2 7})$.

Module 6: Program Correctness

Hoare Triple: $\{P\}S\{Q\}$ means: if precondition P holds and code S executes, then postcondition Q holds upon termination.

Correctness Types:

- **Partial:** If inputs satisfy P and S terminates, then Q holds.
- **Total:** Partial correctness + S terminates for all valid inputs.

Recursive Correctness: To prove a recursive function correct:

1. **Base Case:** Correct for inputs triggering non-recursive paths.
2. **Inductive Step:** Assume recursive calls satisfy specification (Pre $\to$ Post). Show current call satisfies Post.
3. **Termination:** Define a **Monovariant** m : Inputs $\to \mathbb{N}$ (or well-founded set) that strictly decreases with every recursive call.

Iterative Correctness (Loops): For `while G do S`: Define a **Loop Invariant** ϑ.

Proof Obligations:

1. **Initialization:** ϑ holds before the loop starts (ϑ_0).
2. **Maintenance:** If ϑ holds before an iteration and G is true, ϑ holds after the iteration ($\vartheta_k \wedge G \implies \vartheta_{k+1}$).
3. **Termination:** Define a monovariant m (mapping state to $\mathbb{N}$). Show m decreases each iteration and is bounded below.
4. **Postcondition:** Upon exit, invariant holds and guard is false:
$$\vartheta \wedge \neg G \implies Q$$

Example: Division Algorithm To compute $a = qd + r$:

- **Guard:** $r \geq d$
- **Invariant:** $a = qd + r \wedge r \geq 0$
- **Monovariant:** r (strictly decreases, bounded by 0).
- **Exit:** $(a = qd + r \wedge r \geq 0) \wedge \neg(r \geq d) \implies a = qd + r \wedge 0 \leq r < d$.

Module 7: Deterministic Finite Automata

DFA Definition: A 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$:

- Q: Finite set of states.
- Σ: Finite input alphabet.
- $\delta : Q \times \Sigma \to Q$: Transition function.
- $q_0 \in Q$: Start state.
- $F \subseteq Q$: Set of accept states.

Extended Transition Function (δ^*): Describes processing a string $w \in \Sigma^*$. Defined recursively:

$$\delta^*(q, w) = \begin{cases} q & \text{if } w = \epsilon \\ \delta(\delta^*(q, u), a) & \text{if } w = ua \end{cases}$$

(Note: Definition processes string from left to right, though recursive structure can vary).

Language Recognized ($L(M)$): The set of strings accepted by M:

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$

A language is **Recognizable** (or Regular) if there exists a DFA that accepts it.

State Invariants: To prove a DFA is correct, associate a predicate $P_q(w)$ (State Invariant) with each state $q \in Q$.

- $P_q(w)$ is true iff $\delta^*(q_0, w) = q$.
- The set of invariants $\{P_q\}_{q \in Q}$ partitions Σ^*.

Proof by Structural Induction: To show $L(M) = L_{target}$:

1. **Base:** Show $P_{q_0}(\epsilon)$ is true.
2. **Step:** For each transition $\delta(p, a) = q$, show:

$$P_p(u) \implies P_q(ua)$$

3. **Conclusion:** $w \in L(M) \iff \delta^*(q_0, w) \in F \iff \bigvee_{q \in F} P_q(w)$.

Configurations: A snapshot (q, w) where q is current state, w is remaining input. **Computation:** Sequence of configurations $C_0 \to C_1 \to \cdots \to C_n$.

Module 8: Non-Deterministic Finite Automata

NFA Definition: A 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where:

$$\delta : Q \times \Sigma \to \mathcal{P}(Q)$$

ϵ-NFA: Allows transitions on the empty string ϵ.

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(Q)$$

Language Recognized $L(M)$: A word w is accepted if *there exists* at least one path from q_0 to some $q_f \in F$ labeled by w.

$$L(M) = \{w \in \Sigma^* \mid \exists q \in F \text{ s.t. } q \in \delta^*(q_0, w)\}$$

Equivalence of Automata: DFAs, NFAs, and ϵ-NFAs are equivalent in power.

$$\text{DFAs} \subseteq \text{NFAs} \subseteq \epsilon\text{-NFAs}$$

All recognize exactly the class of **Regular Languages**.

Subset Construction (NFA to DFA): Algorithm to convert ϵ-NFA N to DFA D.

- **States (Q_D):** Subsets of Q_N (Power set).
- **Start:** ϵ-closure($\{q_0\}$).
- **Transitions:** For $S \subseteq Q_N$:

$$\delta_D(S, a) = \bigcup_{q \in S} \epsilon\text{-closure}(\delta_N(q, a))$$

- **Final (F_D):** $\{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\}$.

ϵ-closure(S): Set of states reachable from set S using only ϵ-transitions.

State Lower Bounds (Fooling Sets): To prove a language L requires at least n states (or is non-regular): Find a set S of n prefixes such that for any distinct $x, y \in S$, they can be "distinguished" by some suffix z.

$$xz \in L \iff yz \notin L$$

By Pigeonhole Principle, if DFA has $< n$ states, two prefixes map to same state, leading to contradiction.

Module 9: Regular Languages

Closure Properties: If K and L are Regular Languages, then the following are also Regular:

- Union ($K \cup L$)
- Concatenation ($K \cdot L$)
- Kleene Star (K^*)
- Complement ($\Sigma^* \setminus K$)
- Intersection ($K \cap L$)
- Difference ($K \setminus L$)

Regular Expressions (RegEx): Algebraic notation for regular languages. Defined inductively:

1. $\emptyset, \epsilon, a$ (for $a \in \Sigma$) are RegEx.
2. If R, S are RegEx, so are $(R + S)$, (RS), R^*.

RegEx Identities:

- $R + S = S + R$
- $R(S + T) = RS + RT$
- $R + \emptyset = R$, $R\epsilon = R$
- $R\emptyset = \emptyset$
- $(R^*)^* = R^*$
- $\emptyset^* = \epsilon$

Arden's Rule: Useful for solving equations of languages. If $\epsilon \notin L(S)$, then:

$$R = SR + T \iff R = S^*T$$

Pumping Lemma: Used to prove a language is **NOT** regular. If L is regular, $\exists p$ (pumping length) such that $\forall w \in L$ with $|w| \geq p$, w can be split into xyz where:

1. $|y| \geq 1$ (part to pump is non-empty)
2. $|xy| \leq p$ (pumpable part is near start)
3. $\forall i \geq 0, xy^iz \in L$

Proof Strategy (Contradiction): 1. Assume L is regular ($\exists p$). 2. Adversary picks p. 3. You pick specific $w \in L$ (usually using p, e.g., 0^p1^p). 4. Adversary splits $w = xyz$ satisfying constraints. 5. You pick i such that $xy^iz \notin L$. 6. Contradiction $\implies L$ not regular.

Example Non-Regular Languages:

- $EQ = \{0^n1^n \mid n \in \mathbb{N}\}$
- Palindromes
- Balanced Parentheses

Module 10: Equivalence Algorithms

Equivalence Theorem: DFAs, NFAs, ϵ-NFAs, and Regular Expressions (RegEx) are all equivalent in descriptive power. They all recognize the class of Regular Languages.

$$\text{RegEx} \iff \text{NFA} \iff \text{DFA}$$

Generalized NFA (GNFA): An NFA where transitions are labeled with Regular Expressions instead of single symbols. Definition: $G = (Q, \Sigma, r, q_s, q_f)$

- q_s: Start state (no incoming transitions).
- q_f: Final state (no outgoing transitions).
- r: Transition function taking two states to a RegEx.

$$r(q_i, q_j) = R_{ij}$$

Thompson's Algorithm (RegEx $\to$ ϵ-NFA): Recursive construction.

1. **Base ($a \in \Sigma$):** $q_s \xrightarrow{a} q_f$.
2. **Union ($R_1 + R_2$):** New start q_s, final q_f. ϵ-transitions from q_s to starts of N_1, N_2. ϵ-transitions from finals of N_1, N_2 to q_f.
3. **Concat (R_1R_2):** ϵ-transition from final of N_1 to start of N_2.
4. **Star (R^*):** New q_s, q_f. ϵ-transitions: $q_s \to \text{start}(N)$, $\text{final}(N) \to q_f$, $\text{final}(N) \to \text{start}(N)$ (loop), $q_s \to q_f$ (skip).

State Elimination (NFA $\to$ RegEx): Converts a finite automaton to a Regular Expression. **Steps:**

1. **Pre-processing:**
   - Add new start state q_{start} with ϵ-transition to old start.
   - Add new final state q_{accept} with ϵ-transitions from old accept states.
   - Ensure only 2 states remain at the end.
2. **Elimination Loop:** While states remaining > 2:
   - Pick a state k to remove (not start/final).
   - For every pair of states i, j connected via k: Update the transition R_{ij} to include paths going through k:

$$R'_{ij} = R_{ij} + R_{ik}(R_{kk})^*R_{kj}$$

     (Where R_{kk} is a self-loop on k).
   - Remove k.
3. **Result:** Return the expression $R_{start,accept}$ on the single transition remaining.